

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт
з лабораторної роботи №3
з дисципліни: «Архітектура програмного забезпечення»
з теми: «Web застосунок»

Виконав:

ст. гр. ПЗПІ-23-8

Потурайко Н. І.

Перевірив:

асистент кафедри ПІ

Дашенков Д. С.

3 WEB ЗАСТОСУНОК

3.1 Мета роботи

Метою даної лабораторної роботи є проектування, архітектурне планування та програмна розробка повноцінного веб-застосунок для системи інтелектуального моніторингу Aquila. Основним завданням є створення багатофункціональної платформи, що забезпечує взаємодію користувачів та адміністраторів з даними телеметрії через сучасні браузері. Застосунок має реалізовувати складні механізми візуалізації часових рядів (температура, вологість, рівень шуму), надавати інструменти для верифікації тривожних подій через перегляд знімків з камер та забезпечувати повний цикл адміністрування об'єктів. Важливою складовою є впровадження системи інтернаціоналізації та адаптивного дизайну в стилі Cyber-Minimalism, що гарантує стабільну роботу на пристроях з різною роздільною здатністю екрана.

3.2 Хід роботи

3.2.1 Прийняті інженерні рішення

3.2.1.1 Технологічна платформа та вибір архітектури

Для реалізації фронтенд частини обрано сучасний стек технологій на базі Vue.js 3 та інструментарію Vite. Застосунок побудовано як Single Page Application, що дозволяє мінімізувати навантаження на мережу та забезпечити плавність переходів між розділами без повного перезавантаження сторінок. Vite забезпечує швидку збірку модулів через нативні ESM-імпорти, що критично важливо для продуктивності розробки. Архітектура базується на поділі логіки на представлення (Views), компоненти (Components) та сервіси (Services), що відповідає принципам чистого коду та полегшує подальше масштабування системи при додаванні нових типів IoT пристроїв.

3.2.1.2 Реактивність та Composition API

Програмна логіка компонентів реалізована за допомогою Composition API з використанням синтаксису script setup. Це дозволило чітко розмежувати стан

компонентів та їх поведінку. Реактивність забезпечується об'єктами `ref` та `reactive`, що дозволяє веб-клієнту миттєво реагувати на зміну даних у глобальному сховищі або результати мережових запитів. `Computed` властивості використовуються для динамічної фільтрації списків пристроїв та розрахунку середніх показників мікроклімату безпосередньо в браузері користувача, що зменшує кількість звернень до сервера.

3.2.1.3 Глобальне управління станом через `Pinia`

Для централізованого управління даними впроваджено сховище `Pinia`. Воно розділене на декілька модулів: `auth` (зберігання JWT токенів та профілю користувача), `telemetry` (кешування останніх вимірювань) та `settings` (налаштування мови та теми). Такий підхід дозволяє зберігати стан системи між переходами по маршрутах та забезпечує доступ до даних авторизації з будь-якої точки застосунку. `Pinia` інтегрована з `LocalStorage`, що дозволяє зберігати активну сесію та налаштування локалізації навіть після закриття вкладки браузера.

3.2.1.4 Маршрутизація та багаторівневий захист доступу

Навігація організована через `Vue Router` з використанням ієрархічної структури маршрутів. Впроваджено навігаційні гварди (`Navigation Guards`), які перед кожним переходом перевіряють статус авторизації та права доступу. Маршрути адміністратора відокремлені від користувацьких інтерфейсів на рівні конфігурації роутера. Це гарантує, що сторінки управління базами даних та системними логами будуть недоступні для звичайних користувачів навіть за прямої спроби введення адреси в браузері.

3.2.1.5 Мережева інфраструктура та `Axios`

Комунікація з бекендом на `Node.js` реалізована через бібліотеку `Axios` з використанням системи перехоплювачів (`Interceptors`). Перший тип перехоплювачів автоматично додає заголовок `Authorization` з актуальним токеном до кожного вихідного запиту. Другий тип обробляє відповіді сервера, забезпечуючи централізовану обробку помилок (наприклад, `401 Unauthorized` або `403 Forbidden`) та виведення відповідних сповіщень користувачу через систему `Toast-повідомлень`.

Це дозволяє підтримувати надійний зв'язок з API та коректно обробляти завершення сесії.

3.2.1.6 Система інтернаціоналізації (i18n)

Для підтримки багатомовності використано бібліотеку `vue-i18n`. Текстові ресурси розділені за функціональними блоками в окремих JSON файлах для української та англійської локалей. Система підтримує не лише переклад статичного тексту, а й динамічну підстановку параметрів у повідомлення про тривоги. Також реалізовано адаптацію форматів часу та числових значень (температура у Цельсіях, вологість у відсотках) відповідно до обраної мови інтерфейсу, що забезпечує зручність користування для міжнародної аудиторії.

3.2.1.7 Візуалізація даних та графічні компоненти

Інтеграція бібліотеки `Chart.js` дозволила реалізувати високопродуктивні графіки для відображення динаміки змін у приміщеннях. Оскільки система *Aquila* орієнтована на *Cyber-Minimalism*, графіки мають кастомне стилювання: неонові лінії, градієнтні заливки та інтерактивні підказки (tooltips) при наведенні курсора. Компоненти графіків адаптовані для роботи з темною темою, автоматично змінюючи колір осей та сітки при перемиканні режимів оформлення. Реалізовано механізм вибору часового діапазону (остання година, день, тиждень), що ініціює запити до API з відповідними фільтрами дати.

3.2.2 Будова застосунку та взаємодія з системою

3.2.2.1 Функціональні модулі інтерфейсу користувача

Основний робочий простір користувача розділено на інтерактивні зони. Дашборд відображає картки кімнат з живими індикаторами стану (температура, вологість, освітленість). Окремий модуль присвячений візуальній безпеці: галерея знімків завантажує фотофіксації, зроблені пристроями ESP32 під час детекції руху. Кожне зображення супроводжується метаданими: назвою пристрою, часом події та причиною спрацювання тригера. Також реалізовано центр сповіщень, де користувач може переглядати історію алертів та фільтрувати їх за критичністю.

3.2.2.2 Панель адміністрування та управління ресурсами

Адміністративний інтерфейс надає розширені інструменти для управління цілісністю системи. Реалізовано повноцінні CRUD інтерфейси для роботи з обліковими записами користувачів, включаючи можливість їх блокування та зміну ролей. Модуль управління пристроями дозволяє реєструвати нові модулі ESP32 через введення їх унікальної MAC-адреси та призначення їх до конкретних кімнат. Також додано розділ журналів аудиту, де адміністратор може відстежувати всі зміни в базі даних PostgreSQL, що були ініційовані користувачами або автоматичними службами сервера.

3.2.2.3 Обробка та експорт даних

Веб-застосунок забезпечує не лише перегляд, а й базову аналітику даних. Користувачі мають можливість вивантажувати звіти телеметрії за певний період у форматі CSV для подальшого аналізу в зовнішніх системах. При підготовці звіту фронтенд збирає дані з декількох ендпоінтів API, фільтрує їх відповідно до налаштувань користувача та формує файл для завантаження безпосередньо в браузері.

3.3 Висновки

У ході виконання даної лабораторної роботи було спроектовано та програмно реалізовано веб-застосунок для системи Aquila на базі Vue.js 3. Завдяки використанню SPA архітектури вдалося досягти високої швидкості роботи інтерфейсу та забезпечити реактивне оновлення даних моніторингу. Впровадження системи Pinia дозволило ефективно управляти станом авторизації та телеметрії, а інтеграція з Chart.js забезпечила наочність аналізу показників мікроклімату. Реалізована рольова модель доступу та багатомовний інтерфейс роблять систему Aquila готовим рішенням для розгортання в умовах комерційної та приватної експлуатації. Веб-клієнт став сполучною ланкою між IoT обладнанням та кінцевим користувачем, забезпечуючи надійне адміністрування та моніторинг у реальному часі.

ДОДАТОК А

Діаграма прецедентів

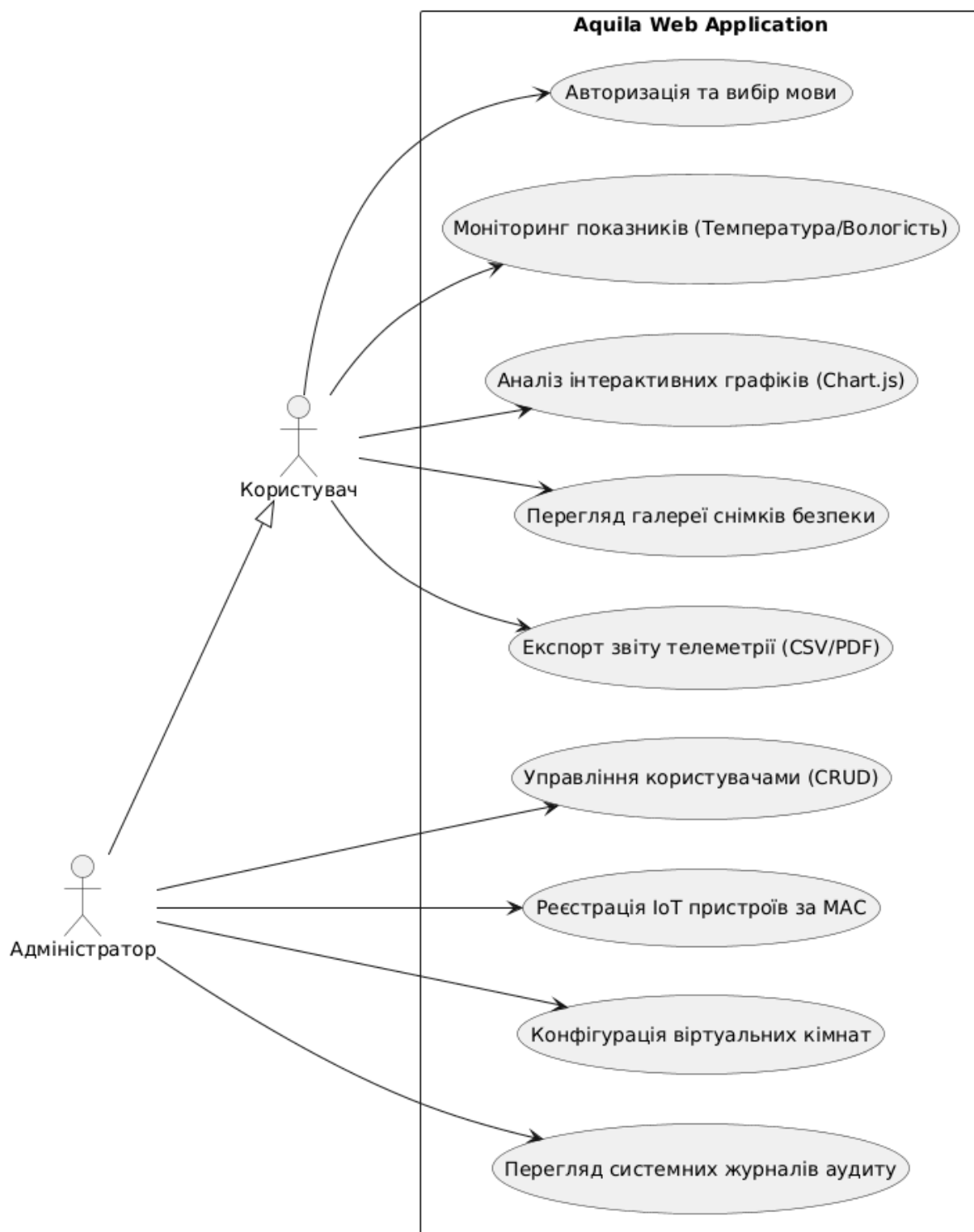


Рисунок А.1 – Сценарії взаємодії адміністраторів та користувачів у веб-інтерфейсі

ДОДАТОК Б

Діаграма компонентів

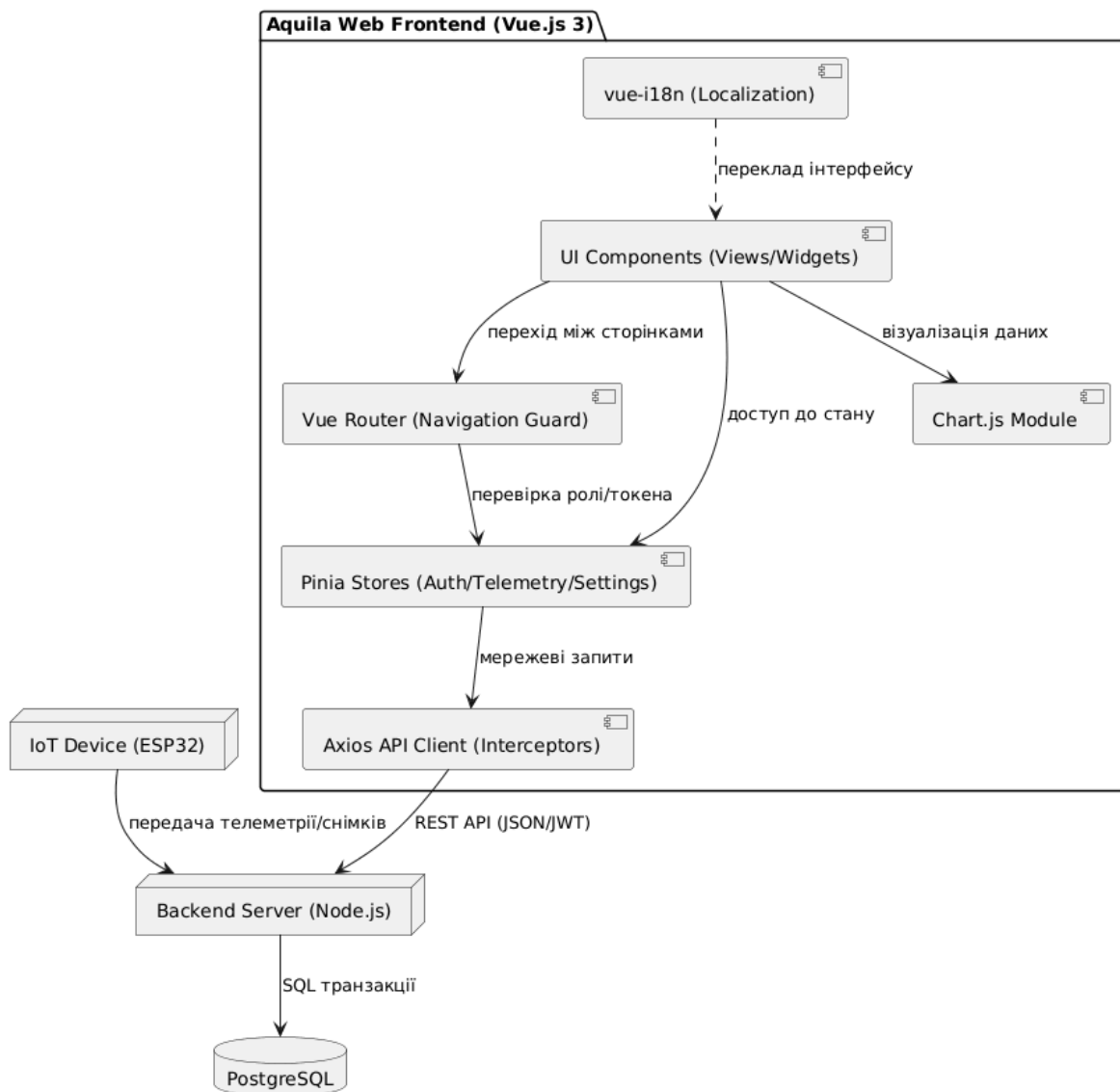


Рисунок Б.1 – Структура Vue.js компонентів та модулів управління станом

ДОДАТОК В

Діаграма діяльності

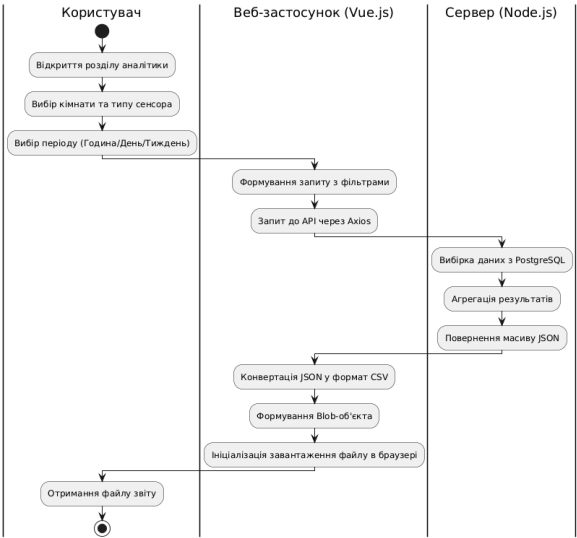


Рисунок В.1 – Алгоритм фільтрації телеметрії та експорту звіту в форматі CSV

ДОДАТОК Г

Діаграма послідовності

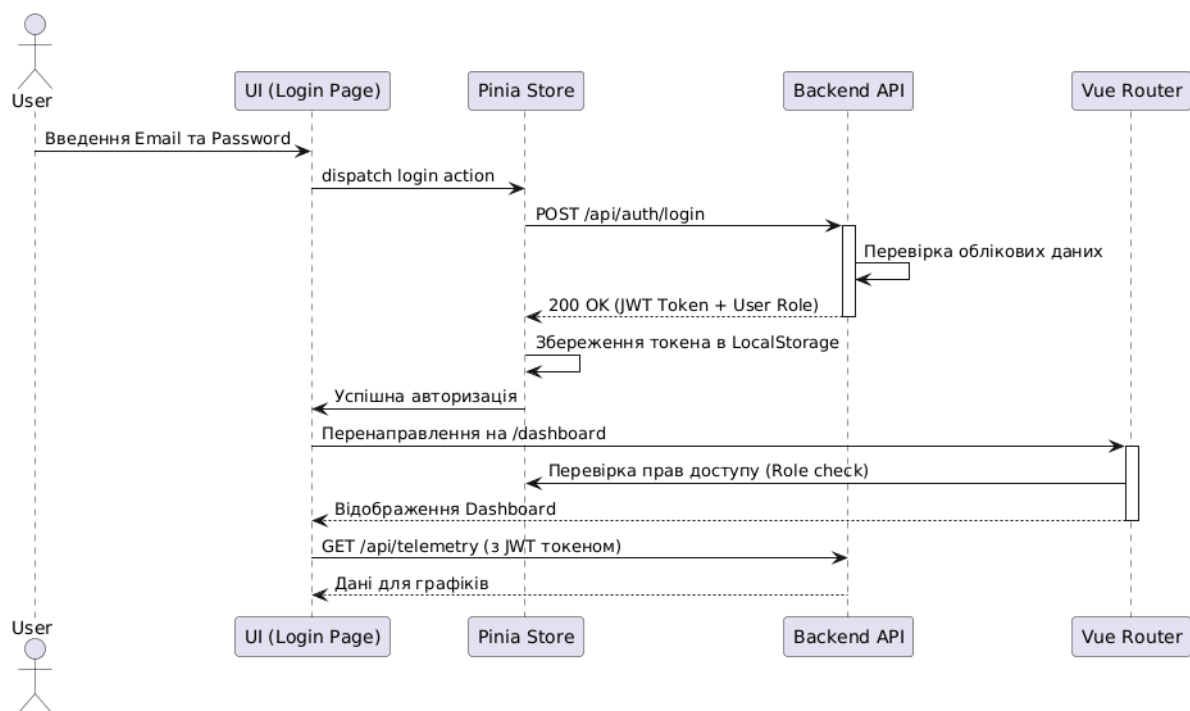


Рисунок Г.1 – Процес авторизації з перевіркою ролі та ініціалізацією мовного пакету

ДОДАТОК Д

Скріншоти інтерфейсу

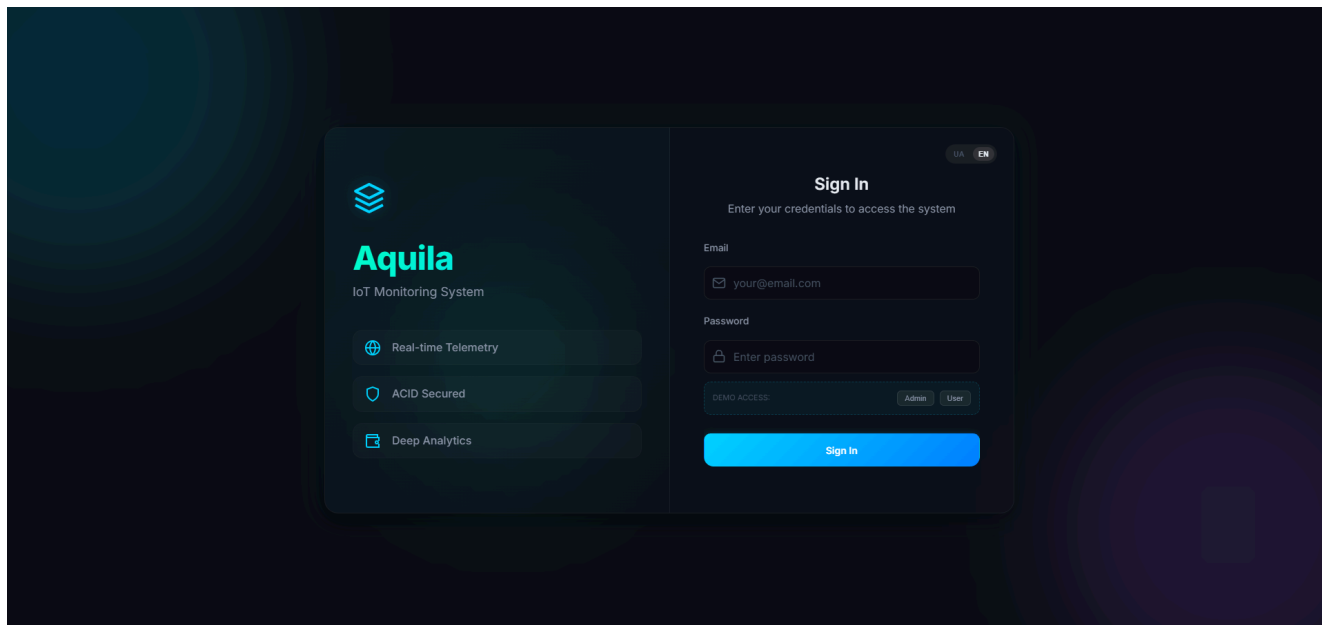


Рисунок Д.1 – Сторінка авторизації

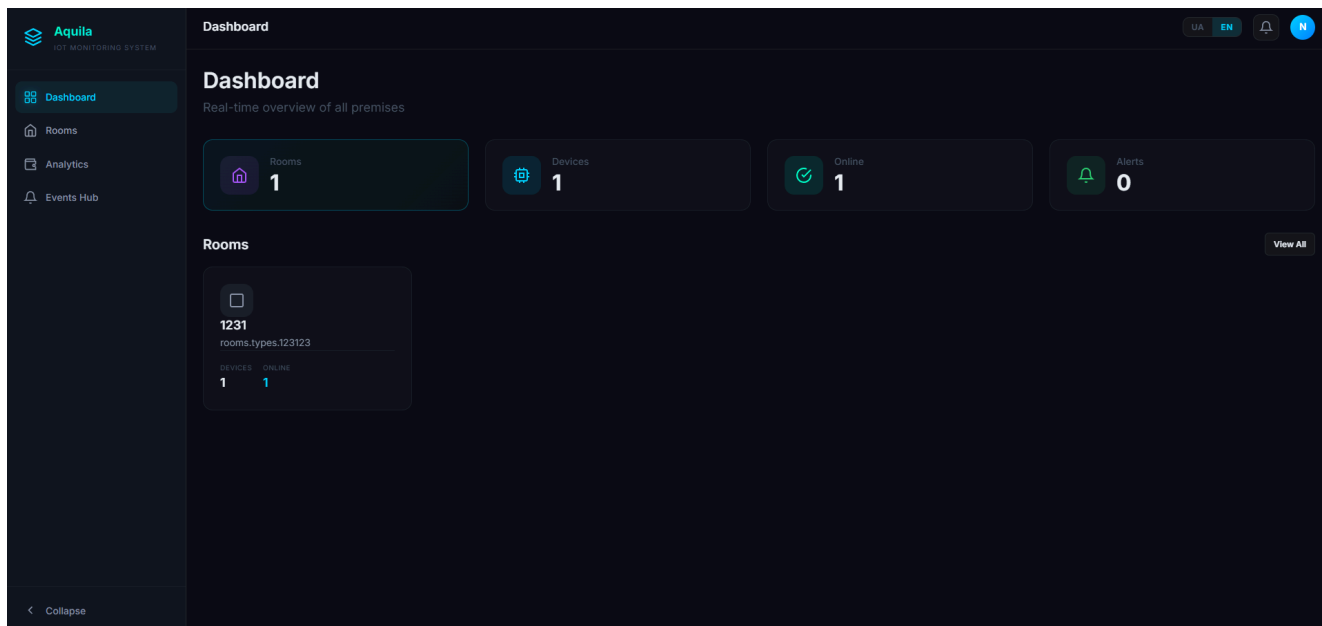


Рисунок Д.2 – Головна панель моніторингу з картками пристроїв та графіками

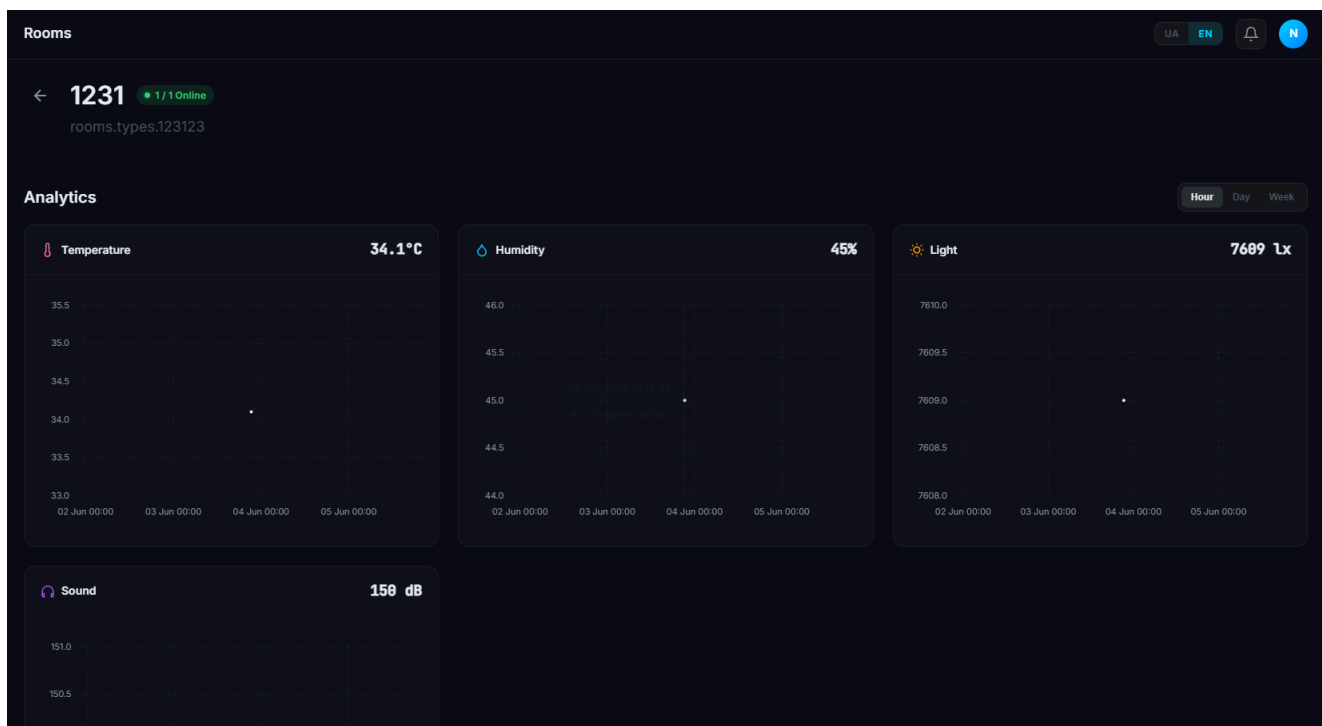


Рисунок Д.3 – Інтерфейс огляду показників у кімнаті

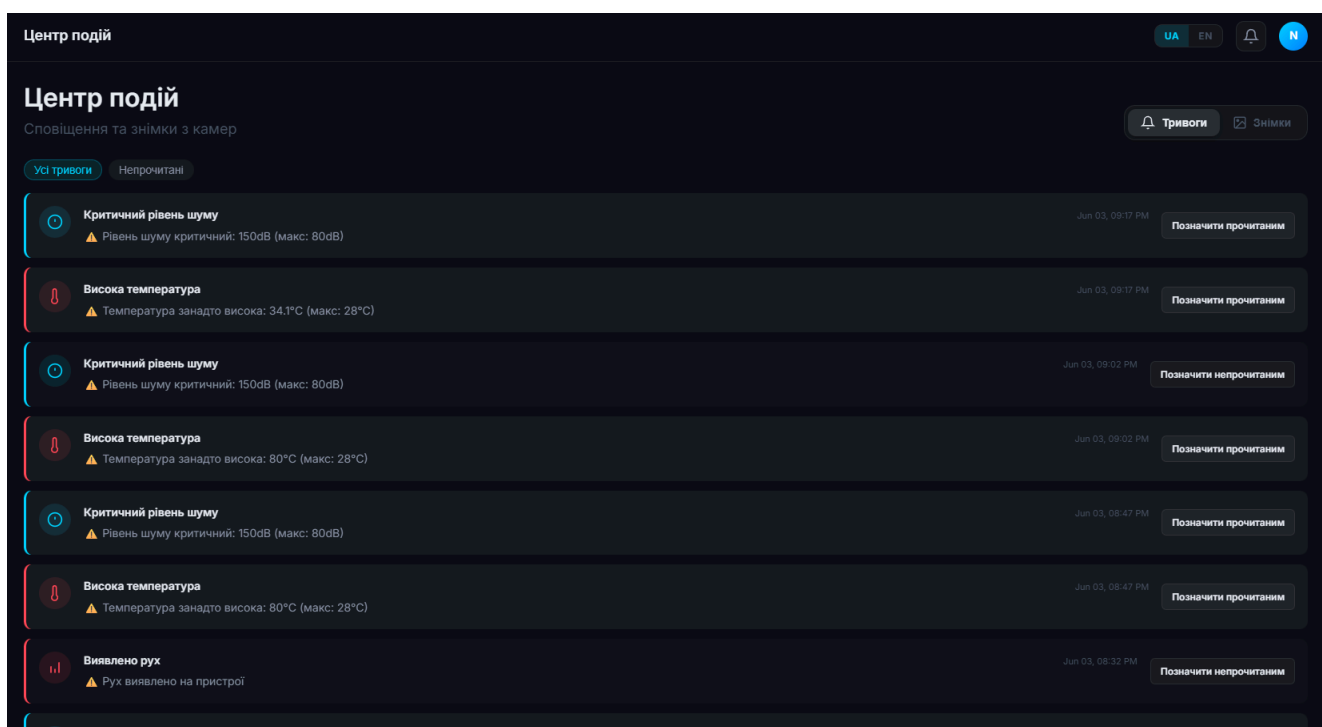


Рисунок Д.4 – Сторінка перегляду сповіщень та знімків

ДОДАТОК Е**Репозиторій**

<https://github.com/NurePoturaikoNazar/apz-project>